Actividad 1: Crear el Microservicio Base

Introducción

Como parte del desarrollo del sistema **Gestión de Citas Inteligente**, se creó un microservicio base utilizando **Spring Boot**, **Java 17** y **PostgreSQL**.
Este microservicio corresponde a la gestión de turnos, ya que representa una de las funcionalidades principales del sistema.

El objetivo de esta actividad fue crear la estructura inicial del microservicio, organizar los paquetes según la arquitectura por capas y configurar la conexión con la base de datos PostgreSQL.

Generación del proyecto en Spring Initializr

Para crear el microservicio base, se utilizó la plataforma **Spring Initializr** disponible en:

https://start.spring.io

La configuración utilizada fue la siguiente:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** versión estable compatible
- **Java:** 17
- **Packaging:** Jar

Dependencias utilizadas

- Spring Web
- Spring Data JPA
- PostgreSQL Driver
- Lombok

Estas dependencias permiten desarrollar una API REST, conectar el microservicio con PostgreSQL y simplificar la escritura de código mediante anotaciones.

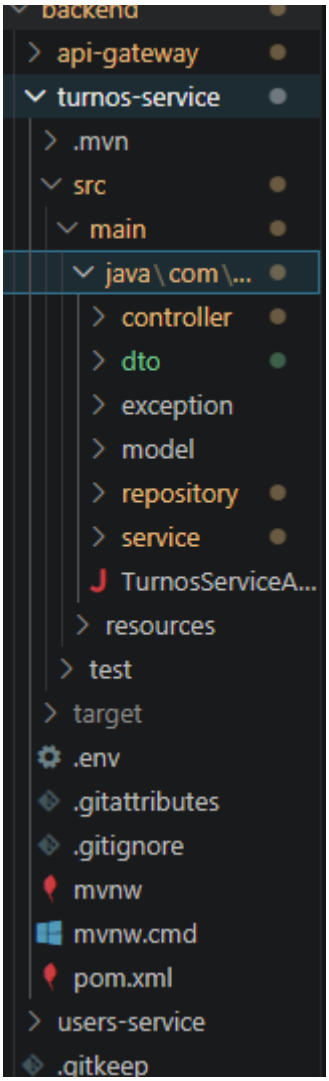
Integración al monorepo

Una vez generado el proyecto, este fue agregado al monorepo del equipo dentro de la carpeta correspondiente al microservicio.

En el proyecto real, la estructura utilizada fue:

turnos-service/

Esto mantiene coherencia con la arquitectura distribuida del sistema, en la cual cada microservicio tiene su propio módulo dentro del mismo repositorio.



Estructura de paquetes

Dentro del microservicio se organizó el código en paquetes, siguiendo el patrón por capas:

- **controller**
- **service**
- **repository**
- **entity**

Descripción de cada paquete

controller:
Contiene los endpoints REST expuestos por el microservicio.

service:
Contiene la lógica de negocio del sistema.

repository:
Permite el acceso a los datos mediante JPA.

entity:
Define las entidades que representan las tablas de la base de datos.

Esta estructura mejora la organización del código, facilita el mantenimiento y permite separar claramente responsabilidades.

Configuración de application.yml

Se configuró el archivo application.yml para conectar el microservicio con PostgreSQL.

Ejemplo de configuración

```
server:
  port: 8082

spring:
  application:
    name: turnos-service

  datasource:
    url: jdbc:postgresql://localhost:5432/turnosdb
    username: postgres
```

```
    password: yeison

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

Explicación

- El microservicio escucha en el puerto **8082**
- Se conecta a la base de datos **turnosdb**
- Se utiliza Hibernate con ddl-auto: update para mantener sincronizada la estructura de tablas
- show-sql: true permite visualizar las consultas generadas

Conclusión Actividad 1

La creación del microservicio base permitió establecer una estructura sólida para el desarrollo del servicio de turnos, integrando Spring Boot con PostgreSQL y organizando el código de forma modular y mantenible.

Actividad 2: Implementar CRUD Completo

Introducción

En esta actividad se implementó el CRUD completo de la entidad principal del microservicio, correspondiente a los **turnos médicos**.
Esto permite crear, consultar, actualizar y eliminar registros desde una API REST.

La entidad principal desarrollada fue:

Turno

Entidad principal

La entidad Turno representa una cita médica dentro del sistema y contiene atributos como:

- idTurno
- doctor
- especialidad
- estado
- fechaCreacion
- fechaHora
- idUsuario

Estos datos permiten registrar la información principal de cada turno.

Endpoints implementados

Se desarrollaron los siguientes endpoints CRUD:

1. Obtener todos los turnos

GET /turnos

Permite consultar todos los registros de turnos almacenados en la base de datos.

2. Obtener un turno por ID

GET /turnos/{id}

Permite consultar un turno específico usando su identificador.

3. Crear un nuevo turno

POST /turnos

Permite registrar una nueva cita médica en el sistema.

4. Actualizar un turno

PUT /turnos/{id}

Permite modificar la información de una cita existente.

5. Eliminar un turno

DELETE /turnos/{id}

Permite eliminar un turno del sistema o cambiar su estado según la lógica implementada.

Ejemplo de request POST

```
{
  "doctor": "Dr. Carlos Perez",
  "especialidad": "Medicina General",
  "estado": "PENDIENTE",
  "fechaHora": "2026-04-30T10:00:00",
  "idUsuario": 1
}
```

Ejemplo de response

```
{
  "idTurno": 8,
  "doctor": "Dr. Carlos Perez",
  "especialidad": "Medicina General",
  "estado": "PENDIENTE",
  "fechaHora": "2026-04-30T10:00:00",
  "idUsuario": 1
}
```

Pruebas con Postman

Se utilizaron herramientas como **Postman** o **Thunder Client** para probar el funcionamiento de todos los endpoints del CRUD.

Las pruebas realizadas permitieron validar:

- creación correcta de turnos
- consulta de todos los registros
- consulta de un turno por ID
- actualización de un registro existente
- eliminación de registros

Estas pruebas confirmaron que el microservicio respondía correctamente ante solicitudes HTTP.

POST http://localhost:8080/turnos

Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "doctor": "Dr. Carlos Perez",
  "especialidad": "Medicina General",
  "estado": "PENDIENTE",
  "fechaHora": "2026-04-30T10:00:00",
  "idUsuario": 1
}
```

Body Cookies Headers (6) Test Results 201 Created 925 ms 400 B Save Response

{} JSON Preview Visualize

```
{
    "idTurno": 8,
    "doctor": "Dr. Carlos Perez",
    "especialidad": "Medicina General",
    "estado": "PENDIENTE",
    "fechaCreacion": "2026-04-13T19:38:57.6002592",
    "fechaHora": "2026-04-30T10:00:00",
    "idUsuario": 1
}
```

Overview New Collection POST Carrito GET Gestion de citas No environment

New Collection / Gestion de citas Save Share

GET http://localhost:8080/turnos

Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

Ctrl+Alt+P to Ask AI

Body Cookies Headers (6) Test Results 200 OK 579 ms 1.67 KB Save Response

{} JSON Preview Visualize


```
[
    {
        "idTurno": 4,
        "doctor": "Dra. Laura Soto",
        "especialidad": "Cardiología",
        "estado": "PENDIENTE",
        "fechaCreacion": "2026-04-12T12:08:38.664837",
        "fechaHora": "2026-04-22T16:00:00",
        "idUsuario": 1
    },
    {
        "idTurno": 5,
        "doctor": "Dra. Camila Torres",
        "especialidad": "Pediatría",
        "estado": "PENDIENTE",
        "fechaCreacion": "2026-04-12T14:09:52.487387",
        "fechaHora": "2026-04-22T08:00:00",
        "idUsuario": 1
```

Verificación en PostgreSQL

Después de realizar las pruebas, se verificaron los datos directamente en PostgreSQL usando la consulta:

```
SELECT * FROM public.turnos
ORDER BY id_turno ASC;
```

Esto permitió confirmar que los cambios realizados desde la API se reflejaban correctamente en la base de datos.

Se observaron registros creados, actualizados y con cambios de estado, lo cual valida el correcto funcionamiento del microservicio.

Data Output Messages Notifications

Showing rows: 1 to 7 Page No: 1 of 1

	id_turno [PK] bigint	doctor character varying (255)	especialidad character varying (255)	estado character varying (255)	fecha_creacion timestamp without time zone (6)	fecha_hora timestamp without time zone (6)	id_usuario bigint
1	1	Dr. Andrés Muñoz	Medicina General	CANCELADO	2026-04-12 11:35:25.80368	2026-04-16 08:00:00	1
2	2	Dr. Andrés Muñoz	Medicina General	CANCELADO	2026-04-12 11:35:29.859933	2026-04-16 08:00:00	1
3	3	Dra. Laura Soto	Cardiología	PENDIENTE	2026-04-12 12:08:28.282062	2026-05-01 09:00:00	1
4	4	Dra. Laura Soto	Cardiología	PENDIENTE	2026-04-12 12:08:38.664837	2026-04-22 16:00:00	1
5	5	Dra. Camila Torres	Pediatría	PENDIENTE	2026-04-12 14:09:52.407387	2026-04-22 08:00:00	1
6	6	Dr. Felipe Ruiz	Cardiología	CANCELADO	2026-04-12 14:14:58.463259	2026-04-21 10:00:00	1
7	7	Dr. Felipe Ruiz	Cardiología	PENDIENTE	2026-04-12 15:27:07.458307	2026-04-25 10:00:00	1

Conclusión Actividad 2

La implementación del CRUD completo permitió disponer de una API funcional para gestionar turnos médicos, integrando correctamente la lógica de negocio, persistencia en base de datos y validación mediante pruebas en Postman y PostgreSQL.

Actividad 3: Hacer Commit y PR

Introducción

Como parte de las buenas prácticas de desarrollo colaborativo, se trabajó utilizando ramas específicas para nuevas funcionalidades, commits descriptivos y Pull Requests para revisión del código antes de integrarlo a la rama principal de desarrollo.

Creación de la feature branch

Se creó una rama específica para desarrollar el primer microservicio, siguiendo la convención propuesta:

feature/SD-XXX-first-microservice

En el contexto del proyecto, también se trabajó con ramas de funcionalidad relacionadas, manteniendo el mismo principio de aislamiento del desarrollo.

Commits descriptivos

Durante el desarrollo del microservicio se realizaron commits descriptivos para cada parte importante del sistema, por ejemplo:

- creación de la entidad
- creación del repositorio
- implementación del servicio
- creación del controlador
- configuración de la base de datos

Este enfoque permite tener un historial claro y entendible de los cambios realizados.

Ejemplos de mensajes de commit:

feat: crear entidad Turno
feat: crear repositorio TurnoRepository
feat: implementar servicio de turnos
feat: crear TurnoController con CRUD
chore: configurar conexión PostgreSQL en application.yml

Push y Pull Request hacia develop

Después de completar el desarrollo, se realizó el push de la rama al repositorio remoto y se creó un **Pull Request** hacia la rama:

develop

Esto permitió integrar el código de forma ordenada y mantener el flujo de trabajo del equipo.

Code Review y aprobación

Finalmente, un integrante del equipo realizó la revisión del código (**Code Review**) y aprobó el Pull Request, validando que la implementación cumplía con los requisitos técnicos y de calidad definidos para el proyecto.

Este proceso fortalece el trabajo colaborativo y reduce errores antes de integrar cambios al entorno principal de desarrollo.

Conclusión Actividad 3

El uso de feature branches, commits descriptivos, Pull Requests y revisión por compañeros permitió desarrollar el microservicio de manera controlada, colaborativa y alineada con buenas prácticas de ingeniería de software.

Reflexión Final

¿Por qué usamos interfaces para los servicios (IUserService) en lugar de clases directas?

Las interfaces se utilizan porque permiten separar la definición de las operaciones de su implementación.
Esto hace que el sistema sea más flexible, mantenible y fácil de modificar en el futuro.

Por ejemplo, una interfaz como ITurnoService define qué operaciones debe tener el servicio, mientras que una clase como TurnoServiceImpl se encarga de implementarlas.
Esto también facilita realizar pruebas, cambiar implementaciones y aplicar principios de diseño más limpios.

¿Qué ventajas tiene JPA/Hibernate sobre escribir SQL manualmente?

JPA e Hibernate permiten trabajar con objetos Java en lugar de escribir consultas SQL manualmente para cada operación.
Entre sus principales ventajas se encuentran:

- menos código repetitivo
- integración directa con entidades Java
- mayor rapidez de desarrollo
- generación automática de consultas básicas
- mejor mantenimiento del código
- portabilidad entre motores de base de datos

Aunque SQL manual sigue siendo útil en casos específicos, JPA/Hibernate simplifica mucho el desarrollo de aplicaciones empresariales.

¿Cómo se relaciona la estructura Controller , Service, Repository con los principios SOLID?

La estructura **Controller ,Service , Repository** ayuda a separar responsabilidades dentro del sistema.

- **Controller:** recibe las solicitudes HTTP
- **Service:** contiene la lógica de negocio
- **Repository:** accede a la base de datos

Esto se relaciona especialmente con el principio **S** de SOLID, que es **Single Responsibility Principle**, porque cada capa tiene una responsabilidad específica.

También favorece otros principios, como:

- **Open/Closed Principle**, al permitir extender el sistema sin modificar demasiado el código existente
- **Dependency Inversion Principle**, al trabajar con interfaces en lugar de depender directamente de implementaciones concretas

En conjunto, esta arquitectura mejora la claridad, reutilización y mantenibilidad del software.